Caso de estudio TecnoMovil Data

David Gil Del Rio

Valentina García Agudelo

Ingeniería de Software y Datos, Institución Universitaria Digital de Antioquia

PREICA2602B020046: Programación Orientada a Objetos II

Jorge Armando Julio Cruz

5 de noviembre de 2026

Introducción

La Alcaldía de TecnoValle adelanta el proyecto TecnoMovil Data para modernizar su sistema de transporte urbano, que moviliza a más de 450.000 pasajeros al día. La plataforma actual procesa los datos de forma secuencial y con estructuras mutables, lo que provoca demoras de hasta dos horas y errores en los conteos por condiciones de carrera (Institución Universitaria Digital de Antioquia [IU Digital], s. f.).

Frente a este panorama, el presente informe describe el diseño e implementación en Java de un módulo de procesamiento de datos basado en programación funcional. La solución utiliza únicamente la biblioteca estándar del lenguaje y trabaja con datos simulados, que reproducen el comportamiento de los validadores, sensores y aplicaciones descritos en el caso.

El documento se organiza así: primero se presentan los objetivos; luego, en el desarrollo, se explican las decisiones de diseño, las seis tareas de análisis y los conceptos funcionales aplicados, junto con las pruebas realizadas y la organización del trabajo; finalmente se exponen las conclusiones y los enlaces a los repositorios de los integrantes. El código fuente completo se incluye en el Apéndice A.

Objetivos

Objetivo general

Desarrollar en Java un módulo funcional que procese datos simulados de transporte urbano, aplicando funciones puras, colecciones inmutables, expresiones lambda y Streams.

Objetivos específicos

1. Diseñar un modelo de datos inmutable y un generador de registros simulados reproducible.

2. Implementar las seis operaciones de análisis solicitadas mediante pipelines declarativos con Streams.
3. Verificar la corrección, la inmutabilidad y la compatibilidad con paralelización mediante pruebas con datos conocidos.
4. Alojar la solución y el informe en repositorios de GitHub, con trabajo colaborativo entre los integrantes.

Desarrollo

Diseño de la solución

El módulo se desarrolló en Java 21 usando únicamente la biblioteca estándar, sin frameworks ni dependencias externas, como lo exige la actividad. El código se organizó en cuatro paquetes con responsabilidades separadas: `modelo`, que define los datos; `datos`, que los simula; `analisis`, que contiene las seis operaciones solicitadas; y la clase `Main`, que ejecuta el conjunto y muestra los resultados. La Figura 1 presenta esta estructura.

Figura 1

Estructura del proyecto TecnoMovil Data

```
src/tecnomovil/
├── modelo/
│   ├── Accion.java
│   └── Registro.java
├── datos/
│   └── GeneradorDatos.java
├── analisis/
│   └── AnalizadorTransporte.java
├── Main.java
└── PruebasAnalizador.java
```

Nota. Elaboración propia.

Antes de escribir código se tomaron algunas decisiones de diseño, resumidas en la Tabla 1. Todas buscan lo mismo: que las operaciones sean predecibles, que no dependan de un estado compartido y que puedan ejecutarse en paralelo sin riesgo.

Tabla 1

Decisiones de diseño y su justificación

Decisión	Justificación
`Registro` como `record`	Sus campos no pueden modificarse después de crearlo, lo que garantiza inmutabilidad sin código adicional
`Accion` como `enum`	Evita errores de escritura frecuentes al usar texto, como “Entrada” o “entrda”
`timestamp` como `LocalDateTime`	Permite obtener la hora del registro y calcular duraciones entre dos momentos
Métodos estáticos sin atributos	Las operaciones no guardan estado, por lo que dependen solo de sus datos de entrada
Métodos que reciben un `Stream`	Quien los invoca decide si el procesamiento es secuencial o paralelo
Resultados con `Map.copyOf` y `toList()`	Las colecciones devueltas no pueden modificarse
Generador con semilla fija	Los datos son reproducibles y los resultados pueden compararse entre ejecuciones

Nota. Elaboración propia.

Una decisión que merece explicación es que los métodos de análisis reciben un `Stream<Registro>` y no una lista. Con este diseño, el análisis no cambia si se invoca con `registros.stream()` o con `registros.parallelStream()`, lo que cumple el requisito de permitir paralelización. Esto es posible porque las operaciones sobre streams son declarativas: describen qué se quiere obtener y dejan a la plataforma la decisión de cómo repartir el trabajo (Oracle, s. f.; Urma et al., 2018).

Modelo de datos

Cada registro del sistema contiene el identificador del usuario, la ruta, la estación, la acción realizada y el momento en que ocurrió. La acción solo puede tomar dos valores, por lo que se modeló con un `enum` llamado `Accion`, que distingue entre `ENTRADA` y `SALIDA`. Este tipo

incluye el método `desdeTexto`, que convierte los textos “entrada” y “salida” del enunciado al valor correspondiente mediante un stream y lanza una excepción si el texto no es válido.

El registro se implementó como un `record`, como se muestra en la Figura 2.

Figura 2

Definición del registro inmutable

```
public record Registro(
        String idUsuario,
        String ruta,
        String estacion,
        Accion accion,
        LocalDateTime timestamp) {

    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario no puede ser nulo");
        // ... validación equivalente para los demás campos
    }

    public int hora() {
        return timestamp.getHour();
    }

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }
}
```

Nota. Se muestra una versión resumida. El código completo está en el Apéndice A.

Un `record` declara sus campos como `final`, de modo que, una vez creado un registro, ninguno de sus datos puede cambiar. Esta característica responde a una recomendación clásica del diseño en Java: reducir al mínimo la mutabilidad de las clases, porque los objetos inmutables son más simples de razonar y seguros para usar desde varios hilos (Bloch, 2018). El constructor compacto agrega una validación adicional, pues rechaza cualquier campo nulo desde el momento de la creación.

Los métodos `hora()` y `esEntrada()` son funciones puras: solo leen los datos del propio registro y devuelven siempre el mismo resultado. Esto permite usarlos como referencias a métodos (`Registro::hora`, `Registro::esEntrada`) dentro de los pipelines de análisis.

Generador de datos simulados

Como el caso de estudio no proporciona datos reales, se construyó la clase `GeneradorDatos`, que simula un día de operación. Cada usuario realiza entre uno y cuatro viajes y cada viaje produce dos registros: una entrada en la estación de origen y una salida en la de destino, que siempre es posterior en el recorrido de la ruta. El 60 % de los viajes se concentra en las horas pico de la mañana (6 a 8) y de la tarde (17 a 19), de modo que los datos muestren un comportamiento parecido al de un sistema real.

La generación se resolvió con un único pipeline, que se presenta en la Figura 3.

Figura 3

Pipeline de generación de registros

```
public static List<Registro> generar(int numUsuarios, long semilla, LocalDate dia) {
    Random rnd = new Random(semilla);

    return IntStream.rangeClosed(1, numUsuarios)
            .mapToObj(n -> "U" + String.format("%05d", n))
            .flatMap(idUsuario -> viajesDeUsuario(idUsuario, rnd, dia))
            .sorted(Comparator.comparing(Registro::timestamp)
                    .thenComparing(Registro::idUsuario))
            .toList();
}
```

Nota. Elaboración propia. El método `toList()` devuelve una lista inmutable.

El pipeline genera los identificadores de usuario, convierte cada uno en sus registros mediante `flatMap`, ordena el resultado por fecha y hora, y entrega una lista que no puede modificarse. Al usar una semilla fija, dos ejecuciones con los mismos parámetros producen exactamente los mismos datos. Con 1000 usuarios y semilla 42 se obtienen 5058 registros, que son los que se usan en todas las evidencias de este informe.

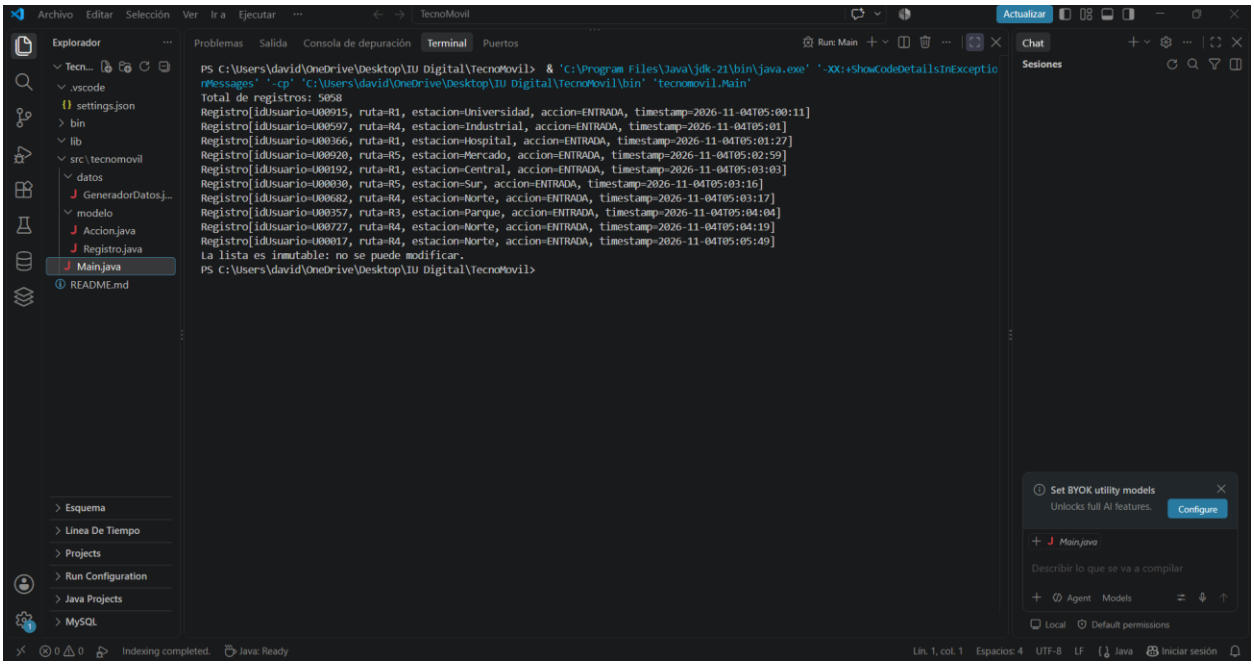
Durante la verificación del generador se detectó un error de diseño: un mismo usuario podía tener dos viajes que se solapaban en el tiempo, lo que producía secuencias imposibles, como entrar a una segunda ruta sin haber salido de la primera. Se corrigió asignando a cada viaje de un usuario una hora distinta y limitando el minuto de inicio, de modo que cada viaje termina

dentro de la misma hora en que empieza. Con este ajuste, las acciones de cada usuario alternan siempre entre entrada y salida.

Conviene aclarar que el generador utiliza la clase `Random`, que mantiene un estado interno y, por tanto, no es una función pura en sentido estricto. Por esa razón se consideró una herramienta de simulación y no parte del módulo de análisis, donde sí se garantiza la ausencia de estado y de efectos secundarios. Por la misma razón el generador se ejecuta de forma secuencial, ya que `Random` no está diseñado para compartirse entre hilos.

Figura 4

Ejecución del generador de datos



Nota. Elaboración propia, a partir de la ejecución en Visual Studio Code.

Implementación de las operaciones de análisis

Las seis operaciones solicitadas se implementaron en la clase `AnalizadorTransporte`, como métodos estáticos que no guardan estado y no modifican sus datos de entrada. Cada método recibe un `Stream<Registro>` y devuelve una colección nueva e inmutable. En esta

sección se presenta cada tarea con su código, la explicación de su pipeline y los resultados obtenidos.

Función base para conteos

Las tareas a), b) y c) comparten la misma lógica: contar cuántos usuarios ingresan, agrupados por algún criterio. En lugar de repetir el código tres veces, se escribió un método auxiliar que recibe el criterio de agrupación como parámetro, como se muestra en la Figura 5.

Figura 5

Función de orden superior para contar ingresos

```
private static <K> Map<K, Long> contarIngresosPor(
        Stream<Registro> registros,
        Function<Registro, K> clasificador) {


    return registros
            .filter(Registro::esEntrada)
            .collect(Collectors.collectingAndThen(
                    Collectors.groupingBy(clasificador, Collectors.counting()),
                    Map::copyOf));
}
```

Nota. Elaboración propia. `Map::copyOf` devuelve una copia inmutable del resultado.

Este método es una función de orden superior, porque recibe otra función (`Function<Registro, K>`) como argumento. El pipeline primero conserva solo los registros de entrada, luego los agrupa según el criterio recibido y cuenta los elementos de cada grupo. Cada tarea solo cambia el criterio: `Registro::estacion`, `Registro::hora` o `Registro::ruta` (Urma et al., 2018).

Tarea a): afluencia por estación

Esta tarea cuenta cuántos usuarios ingresan a cada estación durante el día. Se resuelve con una sola línea, porque reutiliza la función base:

```
public static Map<String, Long> afluenciaPorEstacion(Stream<Registro> registros) {
    return contarIngresosPor(registros, Registro::estacion);
}
```

Los resultados se muestran en la Tabla 2. Se registraron 2529 ingresos en total, y la suma de las catorce estaciones coincide con ese valor, lo que confirma que ningún pasajero se pierde ni se cuenta dos veces.

Tabla 2

Estaciones con mayor afluencia

Estación	Ingresos
Norte	270
Parque	263
Mercado	248
Plaza	239
Central	237

Nota. Se muestran las cinco estaciones con más ingresos, de un total de catorce. Elaboración propia.

Tarea b): Identificación de horas pico

Para esta tarea se agruparon los ingresos por hora del día y se seleccionaron las horas de mayor flujo. El método `flujoPorHora` reutiliza la función base con el criterio `Registro::hora`, y `horasPico` ordena el resultado y toma las primeras (Figura 6).

Figura 6

Pipeline para identificar las horas pico

```
public static List<FlujoHora> horasPico(Stream<Registro> registros, int cantidad) {
    return flujoPorHora(registros).entrySet().stream()
            .map(e -> new FlujoHora(e.getKey(), e.getValue()))
            .sorted(Comparator.comparingLong(FlujoHora::ingresos).reversed()
                    .thenComparingInt(FlujoHora::hora))
            .limit(cantidad)
            .toList();
}
```

Nota. Elaboración propia. `FlujoHora` es un `record` con la hora y la cantidad de ingresos.

El pipeline transforma cada entrada del mapa en un `FlujoHora` (`map`), ordena de mayor a menor flujo (`sorted`), conserva las primeras N (`limit`) y entrega una lista inmutable

(`toList`). Si dos horas tienen el mismo flujo, se prefiere la más temprana, de modo que el resultado sea siempre el mismo. La Tabla 3 muestra que las cinco horas de mayor flujo pertenecen a las franjas pico configuradas en el generador, lo que valida el análisis.

Tabla 3

Cinco horas de mayor flujo

Franja	Ingresos
07:00 – 07:59	311
08:00 – 08:59	307
06:00 – 06:59	304
19:00 – 19:59	300
17:00 – 17:59	298

Nota. Elaboración propia.

Tarea c): Rutas utilizadas

Esta tarea sigue la misma estructura que la anterior, pero agrupando por ruta (Figura 7).

Figura 7

Pipeline para determinar las rutas más utilizadas

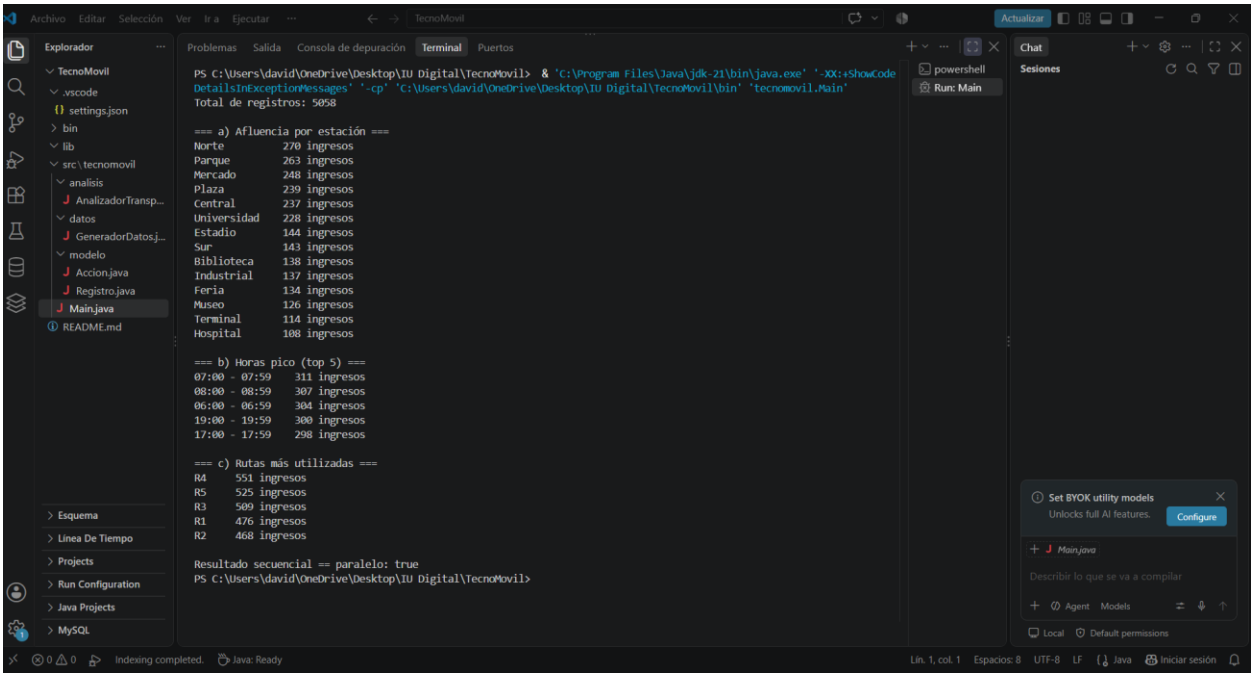
```
public static List<UsoRuta> rutasMasUtilizadas(Stream<Registro> registros, int cantidad) {
    return contarIngresosPor(registros, Registro::ruta).entrySet().stream()
            .map(e -> new UsoRuta(e.getKey(), e.getValue()))
            .sorted(Comparator.comparingLong(UsoRuta::ingresos).reversed()
                    .thenComparing(UsoRuta::ruta))
            .limit(cantidad)
            .toList();
}
```

Nota. Elaboración propia.

La ruta más utilizada fue R4 con 551 ingresos, seguida de R5 (525), R3 (509), R1 (476) y R2 (468). La Figura 8 presenta la ejecución de las tareas a), b) y c).

Figura 8

Ejecución de las tareas a), b) y c)



Nota. Elaboración propia, a partir de la ejecución en Visual Studio Code.

Tarea d): patrones de viaje por usuario

Esta tarea genera, para cada usuario, la lista de estaciones visitadas en el orden en que aparecen sus registros (Figura 9).

Figura 9

Pipeline para obtener los patrones de viaje

```
public static Map<String, List<String>> patronesDeViaje(Stream<Registro> registros) {
    return registros.collect(Collectors.collectingAndThen(
            Collectors.groupingBy(
                    Registro::idUsuario,
                    Collectors.mapping(Registro::estacion, Collectors.toUnmodifiableList())),
            Map::copyOf));
}
```

Nota. Elaboración propia.

El colector `groupingBy` agrupa los registros por usuario, y `mapping` transforma cada registro en el nombre de su estación mientras agrupa. El resultado tiene inmutabilidad en dos

niveles: el mapa externo, gracias a `Map::copyOf`, y cada lista interna, gracias a `toUnmodifiableList`. Como el stream conserva su orden de aparición, las estaciones de cada usuario quedan en el orden correcto incluso al procesar en paralelo (Oracle, s. f.). Por ejemplo, el usuario U00002 tiene el patrón `[Norte, Terminal]`, es decir, entró por Norte y salió en Terminal.

Tarea e): Tiempo promedio entre estaciones

Esta fue la tarea más elaborada. Se interpretó "estaciones consecutivas" como dos registros consecutivos de un mismo usuario: la entrada a una estación y la salida que le sigue en la misma ruta, que representan un trayecto. El procedimiento tiene cuatro pasos: (a) agrupar los registros por usuario; (b) ordenar los de cada usuario por fecha y hora, sobre una copia; (c) recorrer los pares consecutivos y conservar solo los que forman un trayecto, es decir, una entrada seguida de una salida en la misma ruta; y (d) agrupar los trayectos por tramo (ruta, origen y destino) y calcular el promedio.

Figura 10

Pipeline para calcular el tiempo promedio entre estaciones

```
public static List<TiempoTramo> tiempoPromedioEntreEstaciones(Stream<Registro> registros) {
    return registros
            .collect(Collectors.groupingBy(Registro::idUsuario))
            .values().stream()
            .flatMap(AnalizadorTransporte::trayectosDelUsuario)
            .collect(Collectors.groupingBy(
                    t -> new Tramo(t.ruta(), t.origen(), t.destino()),
                    Collectors.summarizingLong(Trayecto::segundos)))
            .entrySet().stream()
            .map(e -> new TiempoTramo(/* ruta, origen, destino, promedio y cantidad */))
            .sorted(/* por ruta, origen y destino */)
            .toList();
}

private static Stream<Trayecto> trayectosDelUsuario(List<Registro> delUsuario) {
    List<Registro> ordenados = delUsuario.stream()
            .sorted(Comparator.comparing(Registro::timestamp))
            .toList();

    return IntStream.range(0, ordenados.size() - 1)
            .filter(i -> esTrayecto(ordenados.get(i), ordenados.get(i + 1)))
            .mapToObj(i -> crearTrayecto(ordenados.get(i), ordenados.get(i + 1)));
}
```

Nota. Versión resumida. El código completo está en el Apéndice A.

El método `flatMap` convierte la colección de usuarios en un único stream de trayectos. El colector `summarizingLong` calcula el promedio y la cantidad de viajes en un solo paso. Los tiempos se midieron en segundos, como números enteros, y no en decimales. Sumar números decimales en distinto orden, como ocurre al paralelizar, puede producir diferencias mínimas; con enteros, el resultado es exacto y el procesamiento paralelo coincide con el secuencial.

La Tabla 4 muestra algunos tramos de la ruta R1. Los tiempos crecen según la cantidad de estaciones intermedias: entre estaciones contiguas, como Central y Norte, el promedio es cercano a 4,5 minutos, mientras que el trayecto completo de Central a Terminal toma cerca de 19 minutos. Esto es coherente con la simulación, que asigna entre 3 y 6 minutos por tramo.

Tabla 4

Tiempo promedio entre estaciones de la ruta R1

Origen	Destino	Minutos (promedio)	Viajes
Central	Norte	4,46	26
Central	Universidad	9,09	35
Central	Hospital	13,61	28
Central	Terminal	18,88	32
Hospital	Terminal	4,51	108

Nota. Se muestran cinco de los diez tramos de la ruta. Elaboración propia.

Tarea f): detección de sobrecarga en rutas

Para esta tarea se definió la ocupación de una ruta como la mayor cantidad de ingresos que recibe en una misma hora, es decir, su punto de máxima demanda. Si ese valor supera un umbral simulado, la ruta se marca como crítica (Figura 11).

Figura 11

Pipeline para detectar rutas críticas

```
public static List<EstadoRuta> detectarSobrecarga(Stream<Registro> registros, long umbral) {
    return registros
            .filter(Registro::esEntrada)
            .collect(Collectors.groupingBy(
                    Registro::ruta,
                    Collectors.groupingBy(Registro::hora, Collectors.counting())))
            .entrySet().stream()
            .map(e -> estadoDeRuta(e.getKey(), e.getValue(), umbral))
            .sorted(Comparator.comparingLong(EstadoRuta::ocupacionPico).reversed()
                    .thenComparing(EstadoRuta::ruta))
            .toList();
}
```

Nota. Elaboración propia.

El pipeline agrupa los ingresos primero por ruta y luego por hora, y para cada ruta busca la hora más cargada. El umbral es un parámetro del método y no un valor fijo dentro del código, así que la función sigue siendo pura y puede probarse con distintos valores. Con un umbral de 65 ingresos por hora, tres de las cinco rutas resultaron críticas (Tabla 5).

Tabla 5

Estado de las rutas con umbral de 65 ingresos por hora

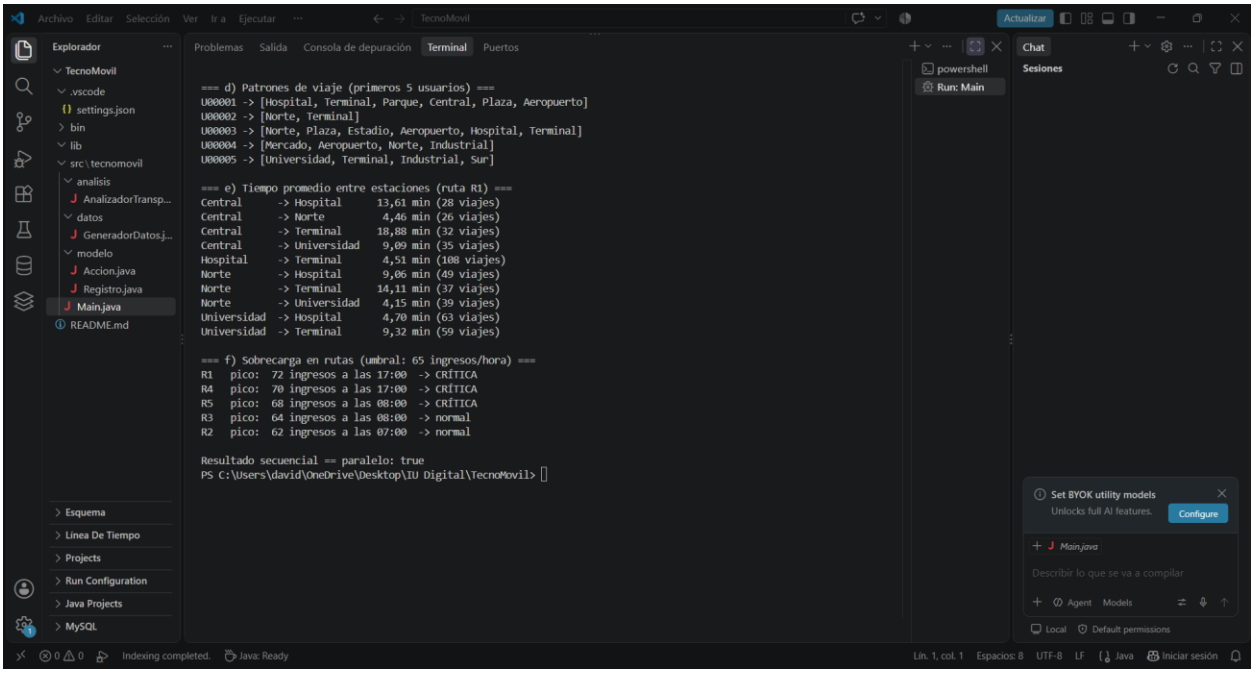
Ruta	Pico de ocupación	Hora	Estado
R1	72	17:00	Crítica
R4	70	17:00	Crítica
R5	68	08:00	Crítica
R3	64	08:00	Normal
R2	62	07:00	Normal

Nota. El umbral es simulado, como lo indica el enunciado. Elaboración propia.

Esta definición de ocupación es una simplificación: mide los ingresos por hora y no la cantidad exacta de pasajeros dentro de los buses en cada instante, dato que el caso de estudio no entrega. Aun así, permite identificar las rutas con mayor presión de demanda. La Figura 12 muestra la ejecución de las tareas d), e) y f).

Figura 12

Ejecución de las tareas d), e) y f)



Nota. Elaboración propia, a partir de la ejecución en Visual Studio Code.

Conceptos de programación funcional aplicados

Esta sección explica cada uno de los conceptos que pedía la actividad y señala dónde se aplican en el código del proyecto.

Funciones puras

Una función pura es aquella que, con los mismos datos de entrada, siempre produce el mismo resultado y no modifica nada fuera de ella, es decir, no tiene efectos secundarios. Los seis métodos de `AnalizadorTransporte` cumplen esta condición: reciben un stream, construyen un resultado nuevo y no leen ni escriben variables externas. El umbral de la tarea f), por ejemplo, entra como parámetro y no como una variable global.

Esta propiedad resuelve el problema descrito en el caso de estudio: el sistema actual comparte estructuras mutables entre operaciones y por eso aparecen errores de conteo por

condiciones de carrera. En la solución funcional no existe un contador compartido; cada colector construye resultados parciales de forma independiente y los combina al final, de modo que el resultado no depende del orden en que se ejecuten las partes (Oracle, s. f.).

Inmutabilidad

La inmutabilidad consiste en que los datos no cambian después de crearse. Se aplicó en tres niveles: en los datos de entrada, con el `record Registro`; en las colecciones auxiliares del generador, con `List.of` y `Map.of`; y en los resultados, con `Map::copyOf`, `toUnmodifiableList` y `toList()`. Además, ninguna operación modifica la lista original: la tarea e), por ejemplo, necesita ordenar los registros de cada usuario y lo hace sobre una copia ordenada (`sorted().toList()`). Las pruebas comprueban que intentar modificar un resultado lanza `UnsupportedOperationException`.

Funciones de orden superior

Una función de orden superior es la que recibe otra función como parámetro o la devuelve. En el proyecto, `contarIngresosPor` recibe un `Function<Registro, K>`, lo que permitió resolver tres tareas con un solo método. También lo son los métodos de la API de streams que se usan en todos los pipelines (`filter`, `map`, `flatMap` y `sorted`) y los comparadores `Comparator.comparing` y `thenComparing`, que reciben funciones para decidir el orden (Urma et al., 2018).

Expresiones lambda y referencias a métodos

Una expresión lambda es una función anónima que se escribe en el lugar donde se necesita. Por ejemplo, `e -> new FlujoHora(e.getKey(), e.getValue())` transforma cada entrada de un mapa en un objeto `FlujoHora`. Cuando la lambda solo llama a un método existente, puede abreviarse como referencia a método: `Registro::estacion` equivale a `r ->`

`r.estacion()`. En el proyecto se usaron ambas formas, y las referencias a métodos hicieron más legibles las agrupaciones y los comparadores.

Streams

Un stream representa una secuencia de elementos sobre la que se aplica un pipeline: una fuente, cero o más operaciones intermedias, que son perezosas, y una operación terminal que produce el resultado (Oracle, s. f.). Este enfoque es declarativo porque se describe qué se quiere obtener y no cómo recorrerlo. La Figura 13 muestra, solo como referencia, cómo se escribiría la tarea a) de forma imperativa.

Figura 13

Equivalente imperativo de la tarea a)

```
Map<String, Long> conteo = new HashMap<>();
for (Registro r : registros) {
    if (r.esEntrada()) {
        long actual = conteo.getOrDefault(r.estacion(), 0L);
        conteo.put(r.estacion(), actual + 1);
    }
}
```

Nota. Elaboración propia. Ejemplo ilustrativo que no forma parte del proyecto.

Frente al pipeline de una línea de la solución, el enfoque imperativo requiere un ciclo, una variable intermedia mutable y un conteo manual, justamente los elementos que provocan errores en el sistema actual. La Tabla 6 resume dónde se aplica cada concepto.

Tabla 6

Conceptos funcionales y su aplicación en el proyecto

Concepto	Dónde se aplica	Ejemplo
Función pura	Los seis métodos de `AnalizadorTransporte`	`afluenciaPorEstacion`
Inmutabilidad	`Registro` y los resultados de cada tarea	`Map::copyOf`, `toList()`
Función de orden superior	Conteos y comparadores	`contarIngresosPor`
Lambda	Operaciones `map`, `filter` y `flatMap`	`e -> new FlujoHora(...)`
Referencia a método	Agrupaciones y comparadores	`Registro::estacion`
Stream	Todas las operaciones	`filter` → `collect`

Nota. Elaboración propia.

Pruebas y paralelización

Pruebas de verificación

Como la actividad no permite frameworks externos, las pruebas se escribieron sin JUnit, en la clase `PruebasAnalizador`. Esta clase trabaja con un conjunto de 12 registros de cinco usuarios, cuyos resultados se calcularon a mano, de modo que cualquier diferencia pueda detectarse con facilidad. Cada prueba es un objeto con un nombre y una condición, y el método `main` las recorre con un stream e imprime si cada una pasó o falló. En total se definieron 18 pruebas, agrupadas como muestra la Tabla 7.

Tabla 7

Pruebas de verificación por grupo

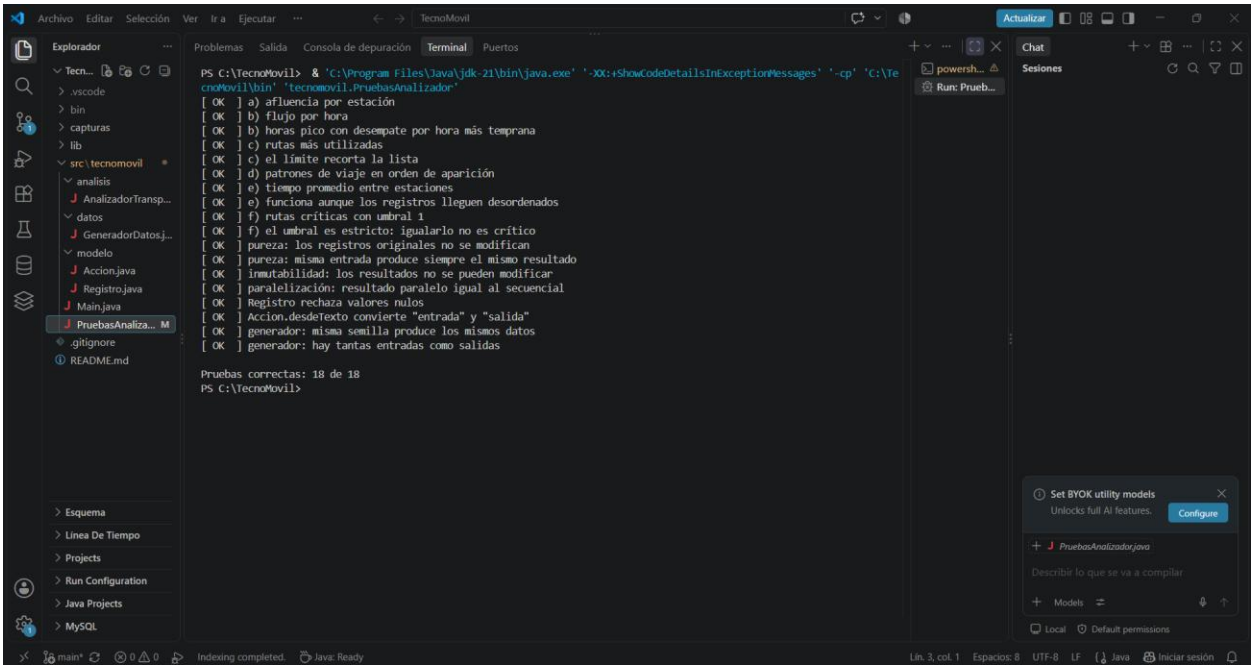
Grupo	Qué verifica	Pruebas
Tareas a), b) y c)	Conteos, orden, desempate por hora y límite de resultados	5
Tareas d), e) y f)	Patrones, promedios, registros desordenados y umbral estricto	5
Pureza e inmutabilidad	Datos originales sin cambios, mismo resultado ante la misma entrada y resultados que no se pueden modificar	3
Paralelización	Resultado paralelo igual al secuencial en las seis tareas	1
Modelo y generador	Rechazo de nulos, conversión de acciones, reproducibilidad y coherencia entre entradas y salidas	4
Total		**18**

Nota. Elaboración propia.

Para confirmar que las pruebas realmente detectan errores, se hizo una verificación adicional: se alteraron a propósito algunos valores esperados y las pruebas correspondientes fallaron, como era de esperar. Con el código original, las 18 pruebas pasan. La Figura 14 presenta la ejecución.

Figura 14

Ejecución de las pruebas de verificación



Nota. Elaboración propia, a partir de la ejecución en Visual Studio Code.

Paralelización

Como los métodos de análisis reciben un `Stream`, la decisión de procesar en paralelo se toma al invocarlos, sin tocar el análisis. La clase `Main` ejecuta las seis tareas de forma secuencial y luego repite el cálculo con `parallelStream()`, comparando ambos resultados con `equals`. Con los 5058 registros del conjunto de datos, los resultados fueron idénticos, y esa misma comprobación forma parte de las pruebas de verificación.

Este resultado se explica por las características del diseño: las funciones son puras, los datos son inmutables y los colectores combinan resultados parciales de forma segura (Oracle, s. f.). Conviene aclarar, sin embargo, que paralelizar no siempre reduce el tiempo de ejecución. Repartir el trabajo entre hilos tiene un costo, y cuando la operación es liviana o el volumen de datos es moderado, ese costo puede superar el beneficio. Por eso la decisión de usar

`parallelStream()` debe apoyarse en mediciones sobre los datos y el equipo reales, algo que en el sistema de TecnoValle, con entre 6 y 12 millones de registros diarios, sí podría justificarse.

Organización del trabajo

El trabajo se repartió entre los dos integrantes como muestra la Tabla 8. El diseño del proyecto, la redacción del informe y la revisión final se hicieron en conjunto, y ambos integrantes revisaron todas las tareas, de modo que cada uno puede explicar la solución completa.

Tabla 8

Distribución de las tareas de análisis

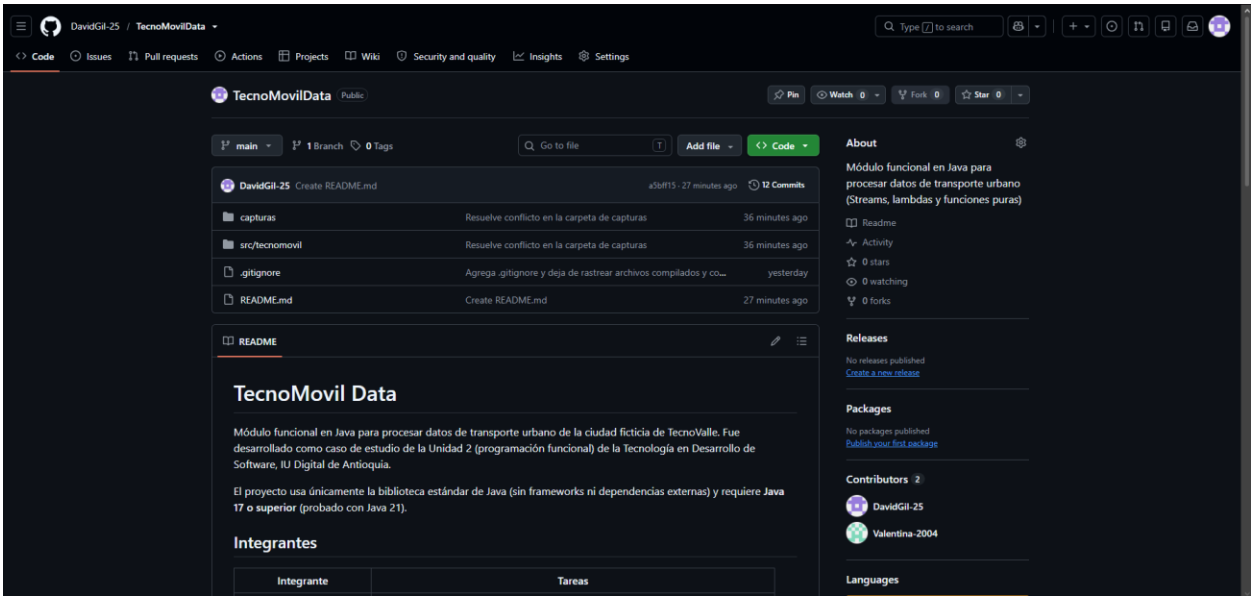
Integrante	Tareas a cargo
David Gil Del Rio	a) Afluencia por estación, b) Horas pico y c) Rutas más utilizadas
Valentina Garcia Agudelo	d) Patrones de viaje, e) Tiempo promedio entre estaciones y f) Sobrecarga en rutas

Nota. Elaboración propia.

La solución se alojó en GitHub, en la rama principal (`main`). Cada repositorio incluye el código fuente, un archivo `README.md` con la descripción del proyecto y la forma de compilarlo y ejecutarlo, y un archivo `.gitignore` que excluye los archivos compilados (`bin/`) y la configuración local del editor (`.vscode/`). El proyecto base se construyó en conjunto y se subió desde el repositorio de David. Los aportes posteriores quedaron registrados mediante commits de cada integrante en su propio repositorio.

Figura 15

Repositorio del proyecto en GitHub



Nota. Elaboración propia, a partir de github.com.

Conclusiones

Se logró diseñar un modelo de datos inmutable y un generador de registros reproducible. El `record Registro` y el `enum Accion` garantizan que los datos no cambien después de su creación, y el uso de una semilla fija permite obtener siempre los mismos 5058 registros a partir de 1000 usuarios, lo que facilitó comparar resultados y validar el análisis.

Las seis operaciones solicitadas se implementaron con pipelines declarativos, sin ciclos ni contadores mutables. La función de orden superior `contarIngresosPor` permitió reutilizar la misma lógica en tres tareas, y la combinación de `flatMap`, `summarizingLong` y colectores anidados resolvió las tareas más elaboradas, como el cálculo de tiempos entre estaciones. Este enfoque produce un código más corto y fácil de leer que el equivalente imperativo, y evita las condiciones de carrera descritas en el caso de estudio.

La corrección del módulo se verificó con 18 pruebas sobre datos calculados a mano. Estas comprobaron los resultados de cada tarea, la pureza de las funciones, la inmutabilidad de los resultados y la equivalencia entre el procesamiento secuencial y el paralelo. Gracias a que los métodos reciben un `Stream`, el módulo puede ejecutarse en paralelo sin modificar el análisis. Con todo, la paralelización no siempre es más rápida, por lo que su uso debe decidirse con mediciones sobre datos y equipos reales.

Finalmente, la solución y el informe se alojaron en repositorios de GitHub, con su documentación y con commits de cada integrante en su propio repositorio, lo que dejó evidencia del proceso de trabajo.

Como limitaciones, la ocupación de las rutas se estimó con los ingresos por hora, porque el caso no entrega la cantidad de pasajeros dentro de los buses en cada instante, y los datos utilizados son simulados. Como trabajo futuro, el módulo podría probarse con datos reales, medir su rendimiento frente al sistema actual y calcular la ocupación instantánea de cada bus.

Enlaces del repositorio

Los repositorios de GitHub de cada integrante son los siguientes:

Tabla 9

Repositorios de los integrantes

Integrante	Repositorio
David Gil Del Rio	https://github.com/DavidGil-25/TecnoMovilData
Valentina Garcia Agudelo	https://github.com/Valentina-2004/TecnoMovilData

Nota. Elaboración propia.

Referencias

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Institución Universitaria Digital de Antioquia. (s. f.). *Unidad 2: Caso de estudio* [Material de curso]. Tecnología en Desarrollo de Software.

Oracle. (s. f.). *Package java.util.stream*. En *Java SE 21 & JDK 21: API documentation*. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html

Urma, R.-G., Fusco, M. y Mycroft, A. (2018). *Modern Java in action: Lambdas, streams, functional and reactive programming* (2.ª ed.). Manning Publications.

Apéndice A

Código fuente completo del proyecto

A continuación, se presenta el código fuente completo del proyecto, tal como fue entregado, organizado por archivo. Las rutas corresponden a la carpeta `src/tecnomovil`.

Acción.java

```
package tecnomovil.modelo;

import java.util.Arrays;

/**
 * Tipos de acción posibles en un registro de transporte.
 * Se usa un enum en lugar de String para evitar errores de escritura.
 */
public enum Accion {
    ENTRADA("entrada"),
    SALIDA("salida");

    private final String etiqueta;

    Accion(String etiqueta) {
        this.etiqueta = etiqueta;
    }

    public String etiqueta() {
        return etiqueta;
    }

    /** Convierte "entrada" o "salida" (como en el enunciado) al enum correspondiente. */
    public static Accion desdeTexto(String texto) {
        return Arrays.stream(values())
                .filter(a -> a.etiqueta.equalsIgnoreCase(texto))
                .findFirst()
                .orElseThrow(() -> new IllegalArgumentException("Acción no válida: " + texto));
    }
}
```

Registro.java

```
package tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Registro inmutable generado por validadores, sensores o aplicaciones.
 * Al ser un record, sus campos son final y no pueden modificarse.
 */
public record Registro(
        String idUsuario,
        String ruta,
        String estacion,
        Accion accion,
        LocalDateTime timestamp) {
```

```
    // Constructor compacto: valida que ningún dato llegue nulo.
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario no puede ser nulo");
        Objects.requireNonNull(ruta, "ruta no puede ser nula");
        Objects.requireNonNull(estacion, "estacion no puede ser nula");
        Objects.requireNonNull(accion, "accion no puede ser nula");
        Objects.requireNonNull(timestamp, "timestamp no puede ser nulo");
    }

    /** Hora del día (0-23) en la que ocurrió el registro. */
    public int hora() {
        return timestamp.getHour();
    }

    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }

    public boolean esSalida() {
        return accion == Accion.SALIDA;
    }
}
```

GeneradorDatos.java

```
package tecnomovil.datos;

import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Registro;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Random;
import java.util.stream.IntStream;
import java.util.stream.Stream;

/**
 * Genera datos simulados de viajes. Con la misma semilla produce siempre
 * los mismos registros, lo que hace los resultados reproducibles.
 */
public final class GeneradorDatos {

    // Rutas y sus estaciones, en orden de recorrido (colecciones inmutables).
    private static final Map<String, List<String>> RUTAS = Map.of(
            "R1", List.of("Central", "Norte", "Universidad", "Hospital", "Terminal"),
            "R2", List.of("Central", "Plaza", "Mercado", "Estadio", "Aeropuerto"),
            "R3", List.of("Terminal", "Parque", "Biblioteca", "Museo", "Central"),
            "R4", List.of("Norte", "Industrial", "Feria", "Plaza", "Sur"),
            "R5", List.of("Sur", "Mercado", "Universidad", "Parque", "Aeropuerto"));

    // Lista aparte para tener un orden fijo (el orden de Map.of puede variar entre ejecuciones).
    private static final List<String> NOMBRES_RUTAS = List.of("R1", "R2", "R3", "R4", "R5");

    private static final List<Integer> HORAS_PICO = List.of(6, 7, 8, 17, 18, 19);

    private GeneradorDatos() {
        // Clase de utilidades: no se instancia.
    }
```

```
    /**
     * Genera los registros de un día para la cantidad de usuarios indicada.
     * Cada usuario hace entre 1 y 4 viajes; cada viaje produce una entrada y una salida.
     *
     * @return lista inmutable de registros ordenada por fecha y hora
     */
    public static List<Registro> generar(int numUsuarios, long semilla, LocalDate dia) {
        Random rnd = new Random(semilla);

        return IntStream.rangeClosed(1, numUsuarios)
                .mapToObj(n -> "U" + String.format("%05d", n))
                .flatMap(idUsuario -> viajesDeUsuario(idUsuario, rnd, dia))
                .sorted(Comparator.comparing(Registro::timestamp)
                        .thenComparing(Registro::idUsuario))
                .toList(); // Stream.toList() devuelve una lista inmutable
    }

    private static Stream<Registro> viajesDeUsuario(String idUsuario, Random rnd, LocalDate dia)
{
        int numViajes = 1 + rnd.nextInt(4);
        // Horas distintas por viaje. Como inicia antes del minuto 35 y dura máximo 24 min,
        // termina dentro de la misma hora y los viajes de un usuario nunca se solapan.
        return Stream.generate(() -> elegirHora(rnd))
                .distinct()
                .limit(numViajes)
                .flatMap(hora -> crearViaje(idUsuario, rnd, dia, hora));
    }

    private static Stream<Registro> crearViaje(String idUsuario, Random rnd, LocalDate dia, int
hora) {
        String ruta = NOMBRES_RUTAS.get(rnd.nextInt(NOMBRES_RUTAS.size()));
        List<String> estaciones = RUTAS.get(ruta);

        int origen = rnd.nextInt(estaciones.size() - 1);
        int destino = origen + 1 + rnd.nextInt(estaciones.size() - origen - 1);

        LocalDateTime inicio = dia.atTime(hora, rnd.nextInt(35), rnd.nextInt(60));
        int minutosPorTramo = 3 + rnd.nextInt(4); // entre 3 y 6 minutos por tramo
        LocalDateTime fin = inicio.plusMinutes((long) (destino - origen) * minutosPorTramo);

        return Stream.of(
                new Registro(idUsuario, ruta, estaciones.get(origen), Accion.ENTRADA, inicio),
                new Registro(idUsuario, ruta, estaciones.get(destino), Accion.SALIDA, fin));
    }

    /** El 60 % de los viajes ocurre en horas pico y el resto se reparte entre las 5 y las 22 h.
*/
    private static int elegirHora(Random rnd) {
        return rnd.nextDouble() < 0.6
                ? HORAS_PICO.get(rnd.nextInt(HORAS_PICO.size()))
                : 5 + rnd.nextInt(18);
    }
}
```

AnalizadorTransporte.java

```
package tecnomovil.analisis;

import tecnomovil.modelo.Registro;

import java.time.Duration;
import java.util.Comparator;
```

```
import java.util.List;
import java.util.Map;
import java.util.function.Function;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;


/**
 * Operaciones funcionales de análisis sobre los registros de transporte.
 *
 * Todos los métodos son funciones puras: no modifican la entrada, no guardan estado
 * y no producen efectos secundarios. Reciben un Stream para que quien los llame
 * decida si procesa de forma secuencial (lista.stream()) o paralela (lista.parallelStream()).
 */
public final class AnalizadorTransporte {

    /** Cantidad de pasajeros que ingresaron en una hora del día. */
    public record FlujoHora(int hora, long ingresos) { }

    /** Cantidad de pasajeros que ingresaron a una ruta. */
    public record UsoRuta(String ruta, long ingresos) { }

    /** Tiempo promedio (en minutos) entre dos estaciones consecutivas de una ruta. */
    public record TiempoTramo(String ruta, String origen, String destino,
                              double minutosPromedio, long viajes) { }

    /** Estado de una ruta: su ocupación máxima y si supera el umbral (crítica). */
    public record EstadoRuta(String ruta, long ocupacionPico, int horaPico, boolean critica) { }

    /** Un recorrido de un usuario entre dos estaciones (uso interno). */
    private record Trayecto(String ruta, String origen, String destino, long segundos) { }

    /** Clave de agrupación para un tramo (uso interno). */
    private record Tramo(String ruta, String origen, String destino) { }

    private AnalizadorTransporte() {
        // Clase de utilidades: no se instancia.
    }

    // ---------------------------------------------------------------
    // Función de orden superior reutilizada por las tareas a), b) y c)
    // ---------------------------------------------------------------

    /**
     * Cuenta los ingresos (acciones "entrada") agrupándolos según el criterio recibido.
     * El criterio es una función (Function), por eso este método es de orden superior.
     */
    private static <K> Map<K, Long> contarIngresosPor(
            Stream<Registro> registros,
            Function<Registro, K> clasificador) {

        return registros
                .filter(Registro::esEntrada)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(clasificador, Collectors.counting()),
                        Map::copyOf)); // copia inmutable del resultado
    }

    // ---------------------------------------------------------------
    // a) Afluencia por estación
    // ---------------------------------------------------------------

    /** Cuántos usuarios ingresan a cada estación durante el día. */
    public static Map<String, Long> afluenciaPorEstacion(Stream<Registro> registros) {
```

```
        return contarIngresosPor(registros, Registro::estacion);
    }

    // ------------------------------------------------------------------
    // b) Horas pico
    // ------------------------------------------------------------------

    /** Ingresos agrupados por hora del día (0-23). */
    public static Map<Integer, Long> flujoPorHora(Stream<Registro> registros) {
        return contarIngresosPor(registros, Registro::hora);
    }

    /** Las "cantidad" horas con mayor flujo, de la más concurrida a la menos concurrida. */
    public static List<FlujoHora> horasPico(Stream<Registro> registros, int cantidad) {
        return flujoPorHora(registros).entrySet().stream()
                .map(e -> new FlujoHora(e.getKey(), e.getValue()))
                .sorted(Comparator.comparingLong(FlujoHora::ingresos).reversed()
                        .thenComparingInt(FlujoHora::hora))
                .limit(cantidad)
                .toList();
    }

    // ------------------------------------------------------------------
    // c) Rutas más utilizadas
    // ------------------------------------------------------------------

    /** Las "cantidad" rutas con mayor volumen de uso, de mayor a menor. */
    public static List<UsoRuta> rutasMasUtilizadas(Stream<Registro> registros, int cantidad) {
        return contarIngresosPor(registros, Registro::ruta).entrySet().stream()
                .map(e -> new UsoRuta(e.getKey(), e.getValue()))
                .sorted(Comparator.comparingLong(UsoRuta::ingresos).reversed()
                        .thenComparing(UsoRuta::ruta))
                .limit(cantidad)
                .toList();
    }

    // ------------------------------------------------------------------
    // d) Patrones de viaje por usuario
    // ------------------------------------------------------------------

    /**
     * Para cada usuario, la lista de estaciones en el orden en que aparecen sus registros.
     * Las listas internas y el mapa externo son inmutables.
     */
    public static Map<String, List<String>> patronesDeViaje(Stream<Registro> registros) {
        return registros.collect(Collectors.collectingAndThen(
                Collectors.groupingBy(
                        Registro::idUsuario,
                        Collectors.mapping(Registro::estacion, Collectors.toUnmodifiableList())),
                Map::copyOf));
    }

    // ------------------------------------------------------------------
    // e) Tiempo promedio entre estaciones
    // ------------------------------------------------------------------

    /**
     * Estima cuánto tardan los usuarios en moverse entre estaciones. Para cada usuario toma
     * sus registros en orden cronológico, une cada "entrada" con la "salida" que le sigue
     * (misma ruta) y promedia los tiempos por tramo.
     */
    public static List<TiempoTramo> tiempoPromedioEntreEstaciones(Stream<Registro> registros) {
        return registros
                .collect(Collectors.groupingBy(Registro::idUsuario))
                .values().stream()
                .flatMap(AnalizadorTransporte::trayectosDelUsuario)
```

```
                .collect(Collectors.groupingBy(
                        t -> new Tramo(t.ruta(), t.origen(), t.destino()),
                        Collectors.summarizingLong(Trayecto::segundos)))
                .entrySet().stream()
                .map(e -> new TiempoTramo(
                        e.getKey().ruta(), e.getKey().origen(), e.getKey().destino(),
                        e.getValue().getAverage() / 60.0,
                        e.getValue().getCount()))
                .sorted(Comparator.comparing(TiempoTramo::ruta)
                        .thenComparing(TiempoTramo::origen)
                        .thenComparing(TiempoTramo::destino))
                .toList();
    }

    /** Convierte los registros de un usuario en sus trayectos (entrada seguida de salida). */
    private static Stream<Trayecto> trayectosDelUsuario(List<Registro> delUsuario) {
        List<Registro> ordenados = delUsuario.stream()
                .sorted(Comparator.comparing(Registro::timestamp))
                .toList(); // copia ordenada: la lista original no se toca

        return IntStream.range(0, ordenados.size() - 1)
                .filter(i -> esTrayecto(ordenados.get(i), ordenados.get(i + 1)))
                .mapToObj(i -> crearTrayecto(ordenados.get(i), ordenados.get(i + 1)));
    }

    private static boolean esTrayecto(Registro inicio, Registro fin) {
        return inicio.esEntrada() && fin.esSalida() && inicio.ruta().equals(fin.ruta());
    }

    private static Trayecto crearTrayecto(Registro inicio, Registro fin) {
        return new Trayecto(inicio.ruta(), inicio.estacion(), fin.estacion(),
                Duration.between(inicio.timestamp(), fin.timestamp()).getSeconds());
    }

    // ------------------------------------------------------------------
    // f) Detección de sobrecarga en rutas
    // ------------------------------------------------------------------

    /**
     * Calcula la ocupación pico de cada ruta (la mayor cantidad de ingresos en una misma hora)
     * y la marca como crítica si supera el umbral. Se ordena de la más a la menos ocupada.
     */
    public static List<EstadoRuta> detectarSobrecarga(Stream<Registro> registros, long umbral) {
        return registros
                .filter(Registro::esEntrada)
                .collect(Collectors.groupingBy(
                        Registro::ruta,
                        Collectors.groupingBy(Registro::hora, Collectors.counting())))
                .entrySet().stream()
                .map(e -> estadoDeRuta(e.getKey(), e.getValue(), umbral))
                .sorted(Comparator.comparingLong(EstadoRuta::ocupacionPico).reversed()
                        .thenComparing(EstadoRuta::ruta))
                .toList();
    }

    private static EstadoRuta estadoDeRuta(String ruta, Map<Integer, Long> ingresosPorHora, long
umbral) {
        FlujoHora pico = ingresosPorHora.entrySet().stream()
                .map(e -> new FlujoHora(e.getKey(), e.getValue()))
                .sorted(Comparator.comparingLong(FlujoHora::ingresos).reversed()
                        .thenComparingInt(FlujoHora::hora))
                .findFirst()
                .orElseThrow();
        return new EstadoRuta(ruta, pico.ingresos(), pico.hora(), pico.ingresos() > umbral);
    }
}
```

Main.java

```
package tecnomovil;

import tecnomovil.analisis.AnalizadorTransporte;
import tecnomovil.datos.GeneradorDatos;
import tecnomovil.modelo.Registro;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Map;

public class Main {

    /** Umbral simulado: más de 65 ingresos en una hora hace que una ruta sea crítica. */
    private static final long UMBRAL_CRITICO = 65;

    public static void main(String[] args) {
        List<Registro> registros = GeneradorDatos.generar(1000, 42L, LocalDate.of(2026, 11, 4));
        System.out.println("Total de registros: " + registros.size());

        // a) Afluencia por estación
        System.out.println("\n=== a) Afluencia por estación ===");
        Map<String, Long> afluencia =
AnalizadorTransporte.afluenciaPorEstacion(registros.stream());
        afluencia.entrySet().stream()
                .sorted(Comparator.comparing((Map.Entry<String, Long> e) ->
e.getValue()).reversed()
                        .thenComparing(Map.Entry::getKey))
                .forEach(e -> System.out.printf("%-12s %5d ingresos%n", e.getKey(),
e.getValue()));

        // b) Horas pico
        System.out.println("\n=== b) Horas pico (top 5) ===");
        AnalizadorTransporte.horasPico(registros.stream(), 5)
                .forEach(f -> System.out.printf("%02d:00 - %02d:59  %5d ingresos%n",
                        f.hora(), f.hora(), f.ingresos()));

        // c) Rutas más utilizadas
        System.out.println("\n=== c) Rutas más utilizadas ===");
        AnalizadorTransporte.rutasMasUtilizadas(registros.stream(), 5)
                .forEach(r -> System.out.printf("%-4s %5d ingresos%n", r.ruta(), r.ingresos()));

        // d) Patrones de viaje por usuario
        System.out.println("\n=== d) Patrones de viaje (primeros 5 usuarios) ===");
        AnalizadorTransporte.patronesDeViaje(registros.stream()).entrySet().stream()
                .sorted(Map.Entry.comparingByKey())
                .limit(5)
                .forEach(e -> System.out.println(e.getKey() + " -> " + e.getValue()));

        // e) Tiempo promedio entre estaciones
        System.out.println("\n=== e) Tiempo promedio entre estaciones (ruta R1) ===");
        AnalizadorTransporte.tiempoPromedioEntreEstaciones(registros.stream()).stream()
                .filter(t -> t.ruta().equals("R1"))
                .forEach(t -> System.out.printf("%-12s -> %-12s %6.2f min (%d viajes)%n",
                        t.origen(), t.destino(), t.minutosPromedio(), t.viajes()));

        // f) Detección de sobrecarga en rutas
        System.out.println("\n=== f) Sobrecarga en rutas (umbral: " + UMBRAL_CRITICO + "
ingresos/hora) ===");
        AnalizadorTransporte.detectarSobrecarga(registros.stream(), UMBRAL_CRITICO)
                .forEach(r -> System.out.printf("%-4s pico: %3d ingresos a las %02d:00  -> %s%n",
                        r.ruta(), r.ocupacionPico(), r.horaPico(),
```

```
                        r.critica() ? "CRÍTICA" : "normal"));

        // Verificación: el resultado paralelo debe ser idéntico al secuencial
        boolean igual = afluencia.equals(
                AnalizadorTransporte.afluenciaPorEstacion(registros.parallelStream()))
                && AnalizadorTransporte.horasPico(registros.stream(), 5)
                        .equals(AnalizadorTransporte.horasPico(registros.parallelStream(), 5))
                && AnalizadorTransporte.rutasMasUtilizadas(registros.stream(), 5)

.equals(AnalizadorTransporte.rutasMasUtilizadas(registros.parallelStream(), 5))
                && AnalizadorTransporte.patronesDeViaje(registros.stream())
                        .equals(AnalizadorTransporte.patronesDeViaje(registros.parallelStream()))
                && AnalizadorTransporte.tiempoPromedioEntreEstaciones(registros.stream())

.equals(AnalizadorTransporte.tiempoPromedioEntreEstaciones(registros.parallelStream()))
                && AnalizadorTransporte.detectarSobrecarga(registros.stream(), UMBRAL_CRITICO)

.equals(AnalizadorTransporte.detectarSobrecarga(registros.parallelStream(), UMBRAL_CRITICO));
        System.out.println("\nResultado secuencial == paralelo: " + igual);
    }
}
```

PruebasAnalizador.java

```
package tecnomovil;

import tecnomovil.analisis.AnalizadorTransporte;
import tecnomovil.analisis.AnalizadorTransporte.EstadoRuta;
import tecnomovil.analisis.AnalizadorTransporte.FlujoHora;
import tecnomovil.analisis.AnalizadorTransporte.TiempoTramo;
import tecnomovil.analisis.AnalizadorTransporte.UsoRuta;
import tecnomovil.datos.GeneradorDatos;
import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Registro;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.function.BooleanSupplier;

import static tecnomovil.analisis.AnalizadorTransporte.afluenciaPorEstacion;
import static tecnomovil.analisis.AnalizadorTransporte.detectarSobrecarga;
import static tecnomovil.analisis.AnalizadorTransporte.flujoPorHora;
import static tecnomovil.analisis.AnalizadorTransporte.horasPico;
import static tecnomovil.analisis.AnalizadorTransporte.patronesDeViaje;
import static tecnomovil.analisis.AnalizadorTransporte.rutasMasUtilizadas;
import static tecnomovil.analisis.AnalizadorTransporte.tiempoPromedioEntreEstaciones;

/**
 * Pruebas de verificación sin frameworks externos (no se usa JUnit).
 *
 * Se trabaja con un conjunto pequeño de 12 registros cuyos resultados se calcularon
 * a mano. Cada prueba compara el resultado del analizador con el valor esperado.
 *
 * Datos de prueba (día 2026-11-04):
 *   U1: R1 Central->Norte 07:00-07:05  y  R2 Plaza->Mercado 17:00-17:10
 *   U2: R1 Central->Universidad 07:30-07:40
 *   U3: R1 Norte->Hospital 08:00-08:15
 *   U4: R2 Plaza->Estadio 07:10-07:20
 *   U5: R1 Central->Norte 09:00-09:15
 */
public class PruebasAnalizador {
```

```
    private record Prueba(String nombre, BooleanSupplier verificacion) { }

    private record Resultado(String nombre, boolean correcta) { }

    private static final LocalDate DIA = LocalDate.of(2026, 11, 4);

    private static Registro reg(String usuario, String ruta, String estacion, Accion accion,
String hora) {
        return new Registro(usuario, ruta, estacion, accion, LocalDateTime.of(DIA,
LocalTime.parse(hora)));
    }

    /** Datos de prueba, en orden cronológico. */
    private static final List<Registro> DATOS = List.of(
            reg("U1", "R1", "Central",     Accion.ENTRADA, "07:00"),
            reg("U1", "R1", "Norte",       Accion.SALIDA,  "07:05"),
            reg("U4", "R2", "Plaza",       Accion.ENTRADA, "07:10"),
            reg("U4", "R2", "Estadio",     Accion.SALIDA,  "07:20"),
            reg("U2", "R1", "Central",     Accion.ENTRADA, "07:30"),
            reg("U2", "R1", "Universidad", Accion.SALIDA,  "07:40"),
            reg("U3", "R1", "Norte",       Accion.ENTRADA, "08:00"),
            reg("U3", "R1", "Hospital",    Accion.SALIDA,  "08:15"),
            reg("U5", "R1", "Central",     Accion.ENTRADA, "09:00"),
            reg("U5", "R1", "Norte",       Accion.SALIDA,  "09:15"),
            reg("U1", "R2", "Plaza",       Accion.ENTRADA, "17:00"),
            reg("U1", "R2", "Mercado",     Accion.SALIDA,  "17:10"));

    private static final List<TiempoTramo> TIEMPOS_ESPERADOS = List.of(
            new TiempoTramo("R1", "Central", "Norte", 10.0, 2),
            new TiempoTramo("R1", "Central", "Universidad", 10.0, 1),
            new TiempoTramo("R1", "Norte", "Hospital", 15.0, 1),
            new TiempoTramo("R2", "Plaza", "Estadio", 10.0, 1),
            new TiempoTramo("R2", "Plaza", "Mercado", 10.0, 1));

    // ------------------------------------------------------------------
    // Utilidades de prueba
    // ------------------------------------------------------------------

    /** Devuelve true si la acción lanza la excepción esperada. */
    private static boolean lanza(Class<? extends Throwable> tipo, Runnable accion) {
        try {
            accion.run();
            return false;
        } catch (Throwable e) {
            return tipo.isInstance(e);
        }
    }

    private static boolean ejecutar(Prueba prueba) {
        try {
            return prueba.verificacion().getAsBoolean();
        } catch (RuntimeException e) {
            return false; // una excepción inesperada cuenta como prueba fallida
        }
    }

    // ------------------------------------------------------------------
    // Catálogo de pruebas
    // ------------------------------------------------------------------

    private static List<Prueba> pruebas() {
        return List.of(
                // --- Tareas a), b) y c) ---
                new Prueba("a) afluencia por estación",
```

```
                        () -> afluenciaPorEstacion(DATOS.stream())
                                .equals(Map.of("Central", 3L, "Norte", 1L, "Plaza", 2L))),

                new Prueba("b) flujo por hora",
                        () -> flujoPorHora(DATOS.stream())
                                .equals(Map.of(7, 3L, 8, 1L, 9, 1L, 17, 1L))),

                new Prueba("b) horas pico con desempate por hora más temprana",
                        () -> horasPico(DATOS.stream(), 3)
                                .equals(List.of(new FlujoHora(7, 3), new FlujoHora(8, 1), new
FlujoHora(9, 1)))),

                new Prueba("c) rutas más utilizadas",
                        () -> rutasMasUtilizadas(DATOS.stream(), 5)
                                .equals(List.of(new UsoRuta("R1", 4), new UsoRuta("R2", 2)))),

                new Prueba("c) el límite recorta la lista",
                        () -> rutasMasUtilizadas(DATOS.stream(), 1)
                                .equals(List.of(new UsoRuta("R1", 4)))),

                // --- Tareas d), e) y f) ---
                new Prueba("d) patrones de viaje en orden de aparición",
                        () -> patronesDeViaje(DATOS.stream()).equals(Map.of(
                                "U1", List.of("Central", "Norte", "Plaza", "Mercado"),
                                "U2", List.of("Central", "Universidad"),
                                "U3", List.of("Norte", "Hospital"),
                                "U4", List.of("Plaza", "Estadio"),
                                "U5", List.of("Central", "Norte")))),

                new Prueba("e) tiempo promedio entre estaciones",
                        () ->
tiempoPromedioEntreEstaciones(DATOS.stream()).equals(TIEMPOS_ESPERADOS)),

                new Prueba("e) funciona aunque los registros lleguen desordenados",
                        () -> tiempoPromedioEntreEstaciones(
                                DATOS.stream()

.sorted(Comparator.comparing(Registro::timestamp).reversed()))
                                .equals(TIEMPOS_ESPERADOS)),

                new Prueba("f) rutas críticas con umbral 1",
                        () -> detectarSobrecarga(DATOS.stream(), 1).equals(List.of(
                                new EstadoRuta("R1", 2, 7, true),
                                new EstadoRuta("R2", 1, 7, false)))),

                new Prueba("f) el umbral es estricto: igualarlo no es crítico",
                        () -> detectarSobrecarga(DATOS.stream(), 2).stream()
                                .noneMatch(EstadoRuta::critica)),

                // --- Propiedades de la programación funcional ---
                new Prueba("pureza: los registros originales no se modifican",
                        () -> {
                            List<Registro> original = new ArrayList<>(DATOS);
                            List<Registro> copia = List.copyOf(original);
                            afluenciaPorEstacion(original.stream());
                            horasPico(original.stream(), 3);
                            rutasMasUtilizadas(original.stream(), 3);
                            patronesDeViaje(original.stream());
                            tiempoPromedioEntreEstaciones(original.stream());
                            detectarSobrecarga(original.stream(), 1);
                            return original.equals(copia);
                        }),

                new Prueba("pureza: misma entrada produce siempre el mismo resultado",
                        () -> tiempoPromedioEntreEstaciones(DATOS.stream())
```

```
                                .equals(tiempoPromedioEntreEstaciones(DATOS.stream()))
                                && detectarSobrecarga(DATOS.stream(), 1)
                                .equals(detectarSobrecarga(DATOS.stream(), 1))),

                new Prueba("inmutabilidad: los resultados no se pueden modificar",
                        () -> lanza(UnsupportedOperationException.class,
                                () -> afluenciaPorEstacion(DATOS.stream()).put("X", 1L))
                                && lanza(UnsupportedOperationException.class,
                                () -> horasPico(DATOS.stream(), 2).add(new FlujoHora(0, 0)))
                                && lanza(UnsupportedOperationException.class,
                                () -> patronesDeViaje(DATOS.stream()).get("U1").add("X"))
                                && lanza(UnsupportedOperationException.class,
                                () -> tiempoPromedioEntreEstaciones(DATOS.stream()).clear())),

                new Prueba("paralelización: resultado paralelo igual al secuencial",
                        () -> afluenciaPorEstacion(DATOS.stream())
                                .equals(afluenciaPorEstacion(DATOS.parallelStream()))
                                && horasPico(DATOS.stream(), 3)
                                .equals(horasPico(DATOS.parallelStream(), 3))
                                && rutasMasUtilizadas(DATOS.stream(), 3)
                                .equals(rutasMasUtilizadas(DATOS.parallelStream(), 3))
                                && patronesDeViaje(DATOS.stream())
                                .equals(patronesDeViaje(DATOS.parallelStream()))
                                && tiempoPromedioEntreEstaciones(DATOS.stream())
                                .equals(tiempoPromedioEntreEstaciones(DATOS.parallelStream()))
                                && detectarSobrecarga(DATOS.stream(), 1)
                                .equals(detectarSobrecarga(DATOS.parallelStream(), 1))),

                // --- Modelo y generador ---
                new Prueba("Registro rechaza valores nulos",
                        () -> lanza(NullPointerException.class,
                                () -> new Registro(null, "R1", "Central", Accion.ENTRADA,
LocalDateTime.now()))),

                new Prueba("Accion.desdeTexto convierte \"entrada\" y \"salida\"",
                        () -> Accion.desdeTexto("entrada") == Accion.ENTRADA
                                && Accion.desdeTexto("SALIDA") == Accion.SALIDA
                                && lanza(IllegalArgumentException.class, () ->
Accion.desdeTexto("otra"))),

                new Prueba("generador: misma semilla produce los mismos datos",
                        () -> GeneradorDatos.generar(50, 1L,
DIA).equals(GeneradorDatos.generar(50, 1L, DIA))),

                new Prueba("generador: hay tantas entradas como salidas",
                        () -> {
                            List<Registro> datos = GeneradorDatos.generar(200, 3L, DIA);
                            return datos.stream().filter(Registro::esEntrada).count()
                                    == datos.stream().filter(Registro::esSalida).count();
                        }));
    }

    // ------------------------------------------------------------------

    public static void main(String[] args) {
        List<Resultado> resultados = pruebas().stream()
                .map(p -> new Resultado(p.nombre(), ejecutar(p)))
                .toList();

        resultados.forEach(r ->
                System.out.printf("[%s] %s%n", r.correcta() ? " OK  " : "FALLA", r.nombre()));

        long correctas = resultados.stream().filter(Resultado::correcta).count();
        System.out.printf("%nPruebas correctas: %d de %d%n", correctas, resultados.size());
```

```
        if (correctas != resultados.size()) {
            System.exit(1); // código de salida distinto de cero si alguna prueba falla
        }
    }
}
```